

LAB EXPERIMENTS

1. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Gray-scale.

A)Program:-

```
# Experiment: Read an image and convert it to grayscale
# Import required libraries
import cv2
import matplotlib.pyplot as plt

# Step 1: Specify the correct image path
# Replace 'your_image.jpg' with your actual image file name
image_path = '/content/Screenshot 2026-02-22 104814.png'

# Step 2: Read the image
image = cv2.imread(image_path)

# Step 3: Check if image is loaded properly
if image is None:
    print("Error: Image file not found.")
    print("Make sure the image is in the same folder as the program.")
    print("Or provide the full file path correctly.")
else:

    # Step 4: Convert original image from BGR to RGB
    original_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Step 5: Convert image to grayscale
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Step 6: Display images
    plt.figure(figsize=(10,5))

    # Original Image
    plt.subplot(1,2,1)
    plt.imshow(original_image)
    plt.title("Original Image")
    plt.axis("off")
```

```
# Grayscale Image
plt.subplot(1,2,2)
plt.imshow(gray_image, cmap='gray')
plt.title("Grayscale Image")
plt.axis("off")

# Show output
plt.show()

# Step 7: Save grayscale image
cv2.imwrite("grayscale_output.jpg", gray_image)

print("Image successfully converted to grayscale.")
```

Output:

Original Image



Grayscale Image



Image successfully converted to grayscale.

2. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Blur using Gaussian Blur.

A)Program:-

```
# Experiment: Read an image and apply Gaussian Blur
# Import required libraries
import cv2
```

```
import matplotlib.pyplot as plt

# Step 1: Specify image path
image_path = '/content/W.jpg'

# Step 2: Read the image
image = cv2.imread(image_path)

# Step 3: Check if image is loaded successfully
if image is None:
    print("Error: Image file not found.")
else:

    # Step 4: Convert image from BGR to RGB for proper display
    original_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Step 5: Apply Gaussian Blur
    # (5,5) is kernel size, 0 means automatic sigma calculation
    blurred_image = cv2.GaussianBlur(original_image, (5,5), 0)

    # Step 6: Display original and blurred images
    plt.figure(figsize=(10,5))

    # Original Image
    plt.subplot(1,2,1)
    plt.imshow(original_image)
    plt.title("Original Image")
    plt.axis("off")

    # Blurred Image
    plt.subplot(1,2,2)
    plt.imshow(blurred_image)
    plt.title("Gaussian Blurred Image")
    plt.axis("off")

    # Show images
    plt.show()

    # Step 7: Save blurred image
    blurred_bgr = cv2.cvtColor(blurred_image, cv2.COLOR_RGB2BGR)
```

```
cv2.imwrite("gaussian_blur_output.jpg", blurred_bgr)
```

```
print("Gaussian blurred image saved successfully.")
```

Output:

Original Image



Gaussian Blurred Image



Gaussian blurred image saved successfully.

3. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to show outline using Canny function.

A) Program:

```
# Experiment: Read an image and detect outlines using Canny Edge Detection
```

```
# Import required libraries
```

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
# Step 1: Specify image path
```

```
image_path = '/content/W.jpg'
```

```
# Step 2: Read the image
```

```
image = cv2.imread(image_path)
```

```
# Step 3: Check if image is loaded properly
```

```
if image is None:
```

```
    print("Error: Image file not found.")
```

```
else:
```

```
# Step 4: Convert image to RGB for display
```

```
original_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```



```
# Step 5: Convert image to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Step 6: Apply Gaussian Blur to reduce noise
blurred_image = cv2.GaussianBlur(gray_image, (5,5), 0)

# Step 7: Apply Canny Edge Detection
edges = cv2.Canny(blurred_image, 100, 200)

# Step 8: Display original and outlined images
plt.figure(figsize=(10,5))

# Original Image
plt.subplot(1,2,1)
plt.imshow(original_image)
plt.title("Original Image")
plt.axis("off")

# Edge Detected Image
plt.subplot(1,2,2)
plt.imshow(edges, cmap='gray')
plt.title("Canny Outline Image")
plt.axis("off")

# Show output
plt.show()

# Step 9: Save edge-detected image
cv2.imwrite("canny_outline_output.jpg", edges)

print("Outline image saved successfully.")
```

Output:

Original Image



Canny Outline Image



Outline image saved successfully.

4. Implement histogram equalization on the given image and compare it with the original image using Open CV.

A)Program:

```
# Experiment: Histogram Equalization using OpenCV
```

```
# Import required libraries
```

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
# Step 1: Specify image path
```

```
image_path = '/content/JAS.jpg'
```

```
# Step 2: Read image in grayscale mode
```

```
image = cv2.imread(image_path, 0)
```

```
# Step 3: Check if image is loaded properly
```

```
if image is None:
```

```
    print("Error: Image file not found.")
```

```
else:
```

```
    # Step 4: Apply Histogram Equalization
```

```
    equalized_image = cv2.equalizeHist(image)
```

```
# Step 5: Display original and equalized images
```

```
plt.figure(figsize=(12,6))
```

```
# Original Image
```

```
plt.subplot(2,2,1)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.axis("off")

# Equalized Image
plt.subplot(2,2,2)
plt.imshow(equalized_image, cmap='gray')
plt.title("Histogram Equalized Image")
plt.axis("off")

# Original Histogram
plt.subplot(2,2,3)
plt.hist(image.ravel(), 256, [0,256])
plt.title("Original Histogram")

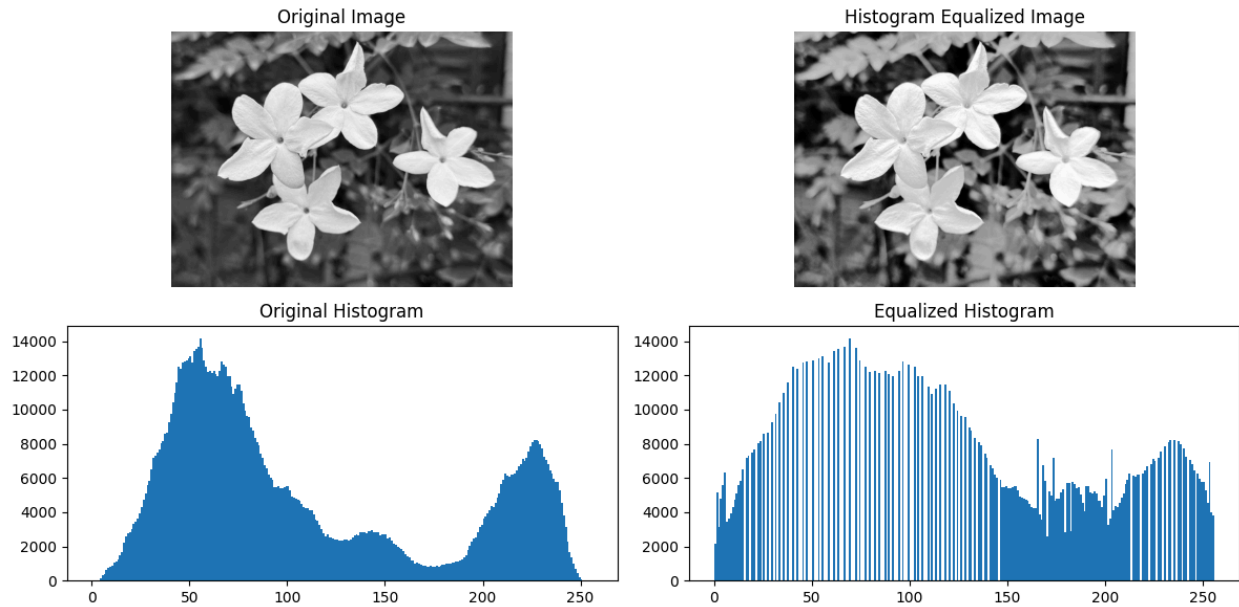
# Equalized Histogram
plt.subplot(2,2,4)
plt.hist(equalized_image.ravel(), 256, [0,256])
plt.title("Equalized Histogram")

# Show results
plt.tight_layout()
plt.show()

# Step 6: Save equalized image
cv2.imwrite("equalized_output.jpg", equalized_image)

print("Histogram equalized image saved successfully.")
```

Output:



Histogram equalized image saved successfully.

5. Write a Python function to analyze the histogram of the given input image based on color levels using Open CV.

A)Program:

Experiment: Analyze histogram of an image based on color levels

Import required libraries

import cv2

import matplotlib.pyplot as plt

Step 1: Define histogram analysis function

def analyze_histogram(image_path):

 # Step 2: Read the image

 image = cv2.imread(image_path)

 # Step 3: Check if image is loaded properly

 if image is None:

 print("Error: Image file not found.")

 return

 # Step 4: Convert image from BGR to RGB

 image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

```
# Step 5: Define colors for channels
colors = ('b', 'g', 'r')
```

```
# Step 6: Display original image
plt.figure(figsize=(12,6))
```

```
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
```

```
# Step 7: Plot histogram for each channel
plt.subplot(1,2,2)
```

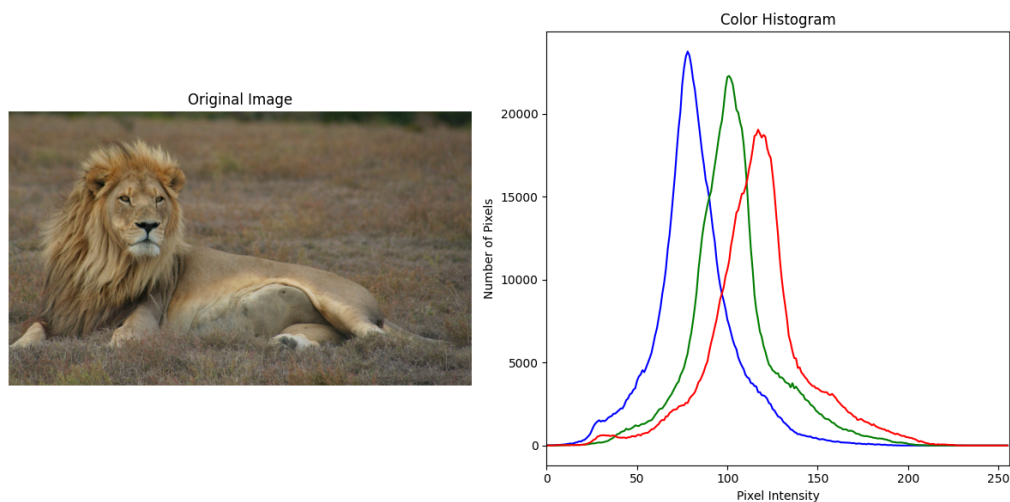
```
for i, color in enumerate(colors):
    hist = cv2.calcHist([image], [i], None, [256], [0,256])
    plt.plot(hist, color=color)
    plt.xlim([0,256])
```

```
plt.title("Color Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Number of Pixels")
```

```
# Step 8: Show results
plt.tight_layout()
plt.show()
```

```
# Step 9: Call function
analyze_histogram('/content/L-IMAGE.jpg')
```

Output:



6. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to erode using Erode function.

A)Program:

Experiment: Read an image and apply erosion

Import required libraries

import cv2

import matplotlib.pyplot as plt

import numpy as np

Step 1: Specify image path

image_path = '/content/L-IMAGE.jpg'

Step 2: Read the image

image = cv2.imread(image_path)

Step 3: Check if image is loaded successfully

if image is None:

 print("Error: Image file not found.")

else:

Step 4: Convert image from BGR to RGB for display

original_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

Step 5: Create kernel for erosion

kernel = np.ones((5,5), np.uint8)

Step 6: Apply erosion

eroded_image = cv2.erode(image, kernel, iterations=1)

Step 7: Convert eroded image for display

eroded_display = cv2.cvtColor(eroded_image, cv2.COLOR_BGR2RGB)

Step 8: Display original and eroded images

plt.figure(figsize=(10,5))

Original Image

plt.subplot(1,2,1)

plt.imshow(original_image)

plt.title("Original Image")

```
plt.axis("off")

# Eroded Image
plt.subplot(1,2,2)
plt.imshow(eroded_display)
plt.title("Eroded Image")
plt.axis("off")

# Show output
plt.show()

# Step 9: Save eroded image
cv2.imwrite("eroded_output.jpg", eroded_image)

print("Eroded image saved successfully.")
```

Output:

Original Image



Eroded Image



Eroded image saved successfully.

7. Perform basic video processing operations on the captured video. Read captured video in python and display the video, in slow motion and in fast motion.

A) Program:

```
import cv2
from google.colab.patches import cv2_imshow

# Read video
cap = cv2.VideoCapture('/content/171227-845081742_medium.mp4')

while cap.isOpened():
    ret, frame = cap.read()
```

```
if not ret:  
    break  
  
# Display frame  
cv2.imshow('frame', frame)  
  
# Slow motion → increase delay  
# Fast motion → decrease delay  
if cv2.waitKey(100) & 0xFF == 27: # try 10 for fast  
    break
```

```
cap.release()  
cv2.destroyAllWindows()
```

Output:



The captured video is successfully read and displayed as frames, demonstrating slow and fast motion effects by adjusting the frame delay.

8. Perform basic Image Handling and processing operations on the image is to read an image in python and Dilate an Image using Dilate function

A)Program:

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

# Read image
img = cv2.imread('/content/JAS.jpg')

# Check if image loaded
if img is None:
    print("Error: Image not found")
else:
    # Create kernel
    kernel = np.ones((5,5), np.uint8)

    # Apply dilation
    dilated = cv2.dilate(img, kernel, iterations=1)

    # Display images
    cv2_imshow(img)
    cv2_imshow(dilated)
```

Output:



Original image



Dilated image

9.Implement the image scaling techniques to resize the images to both bigger and small sizes

A)Program:

```
import cv2
```

```
img = cv2.imread('image.jpg')
```

```
# Smaller image
```

```
small = cv2.resize(img, (200, 200))
```

```
# Bigger image
```

```
big = cv2.resize(img, (800, 800))
```

```
cv2.imshow("Original", img)
```

```
cv2.imshow("Small Image", small)
```

```
cv2.imshow("Big Image", big)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Output:

Original Image:



Small Image:



Big Image:



10. Perform a 90-degree rotation clockwise along the y-axis for the given image.

A) Program:

```
import cv2
from google.colab.patches import cv2_imshow
```

```
# Read image
img = cv2.imread('/content/W.jpg')
```

```
# Check if image loaded
if img is None:
```



```
print("Error: Image not found")
else:
    # Rotate image
    rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)

    # Display images
    print("Original Image:")
    cv2_imshow(img)

    print("Rotated Image:")
    cv2_imshow(rotated)
```

Output:
Original Image:



Rotated Image:



